Deep Learning Assignment

SMS Spam Detection using Vanilla RNN (SimpleRNN)

Objective

The objective of this assignment is to build a deep learning model that classifies SMS messages as spam or ham using a Vanilla RNN (SimpleRNN).

The project includes:

- Data cleaning and exploration
- Text preprocessing for sequence models
- Building and training a Vanilla RNN
- Evaluating performance using precision, recall, F1-score, and confusion matrix
- Improving model performance through tuning and error analysis

```
# Core libraries
import pandas as pd
import numpy as np

# Visualization
import matplotlib.pyplot as plt
import seaborn as sns

# Sklearn utilities
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report, confusion_matrix

# TensorFlow / Keras
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, SimpleRNN, Dense, Dropout
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences
from tensorflow.keras.callbacks import EarlyStopping
```

```
# TASK -A PART -01
# Load dataset with correct encoding
df = pd.read_csv("spam.csv", encoding="latin-1")

# Display first 5 rows
df.head()
```

	v1	v2	Unnamed: 2	Unnamed: 3	Unnamed: 4
0	ham	Go until jurong point, crazy.. Available only ...	NaN	NaN	NaN
1	ham	Ok lar... Joking wif u oni...	NaN	NaN	NaN
2	spam	Free entry in 2 a wkly comp to win FA Cup fina...	NaN	NaN	NaN
[illegible]	[illegible]	U dun say so early hor... U c	[illegible]	[illegible]	[illegible]

Next steps: Generate code with df New interactive sheet

```
# TASK -A PART -02
# Keep only required columns
df = df[['v1', 'v2']]

# Display first rows again
df.head()
```

	v1	v2
0	ham	Go until jurong point, crazy.. Available only ...
1	ham	Ok lar... Joking wif u oni...
2	spam	Free entry in 2 a wkly comp to win FA Cup fina...
3	ham	U dun say so early hor... U c already then say...
4	ham	Nah I don't think he goes to usf, he lives aro...

Next steps: Generate code with df New interactive sheet

```
# TASK -A PART -03

# Rename columns
df.columns = ['label', 'text']

# Check result
df.head()
```

	label	text
0	ham	Go until jurong point, crazy.. Available only ...
1	ham	Ok lar... Joking wif u oni...
2	spam	Free entry in 2 a wkly comp to win FA Cup fina...
3	ham	U dun say so early hor... U c already then say...
4	ham	Nah I don't think he goes to usf, he lives aro...

Next steps: Generate code with df New interactive sheet

```
# TASK -A PART -04

# Encode labels
df['label'] = df['label'].map({'ham':0, 'spam':1})

# Check result
df.head()
```

	label	text
0	0	Go until jurong point, crazy.. Available only ...
1	0	Ok lar... Joking wif u oni...
2	1	Free entry in 2 a wkly comp to win FA Cup fina...
3	0	U dun say so early hor... U c already then say...
4	0	Nah I don't think he goes to usf, he lives aro...

Next steps: Generate code with df New interactive sheet

```
# TASK -A PART -05

# Dataset shape after cleaning
df.shape
```

```
(5572, 2)
```

```
# TASK -A PART -06

# Class distribution (counts)
df['label'].value_counts()
```

label	count
0	4825
1	747

dtype: int64

```
# TASK -A PART -07

# Class distribution (percentage)
df['label'].value_counts(normalize=True) * 100
```

label	proportion
0	86.593683
1	13.406317

dtype: float64

TASK -A PART -08

Data Summary

The SMS Spam dataset was cleaned and prepared for modeling.

Data Cleaning Steps:

- Only the relevant columns (v1 and v2) were retained.
- Columns were renamed to **label** and **text**.
- Labels were encoded as:
  - ham = 0
  - spam = 1

Dataset Shape:

- The cleaned dataset contains **5572 SMS messages** and **2 columns**.

Class Distribution:

- Ham messages: ~86.6%
- Spam messages: ~13.4%

The dataset is imbalanced, with significantly more ham messages than spam messages.

```
# TASK -B PART-01
# Train / Validation / Test Split (80/10/10 with stratification)

# Separate features and labels
X = df['text']
y = df['label']

# First split: Train (80%) and Temp (20%)
X_train, X_temp, y_train, y_temp = train_test_split(
    X,
    y,
    test_size=0.2,
    stratify=y,
    random_state=42
)
```

```
# Second split: Validation (10%) and Test (10%)
X_val, X_test, y_val, y_test = train_test_split(
    X_temp,
    y_temp,
    test_size=0.5,
    stratify=y_temp,
    random_state=42
)
```

```
#TASK -B PART-02
#Verify Split Sizes
print("Training samples:", len(X_train))
print("Validation samples:", len(X_val))
print("Test samples:", len(X_test))
```

```
Training samples: 4457
Validation samples: 557
Test samples: 558
```

TASK -B PART-03

Train / Validation / Test Split

The dataset was divided into three subsets:

- Training Set (80%) – used to train the model.
- Validation Set (10%) – used to tune model parameters and monitor performance during training.
- Test Set (10%) – used to evaluate the final model performance.

Stratified sampling was used to ensure that the proportion of spam and ham messages remains consistent across all subsets.

```
#TASK -B PART-04
#Tokenize Messages

# Define vocabulary size
vocab_size = 10000

# Create tokenizer
tokenizer = Tokenizer(num_words=vocab_size, oov_token="<OOV>")

# Fit tokenizer on training data only
tokenizer.fit_on_texts(X_train)
```

```
# TASK -B PART-05
# Convert Text to Sequences
```

```
# Convert text to sequences
X_train_seq = tokenizer.texts_to_sequences(X_train)
X_val_seq = tokenizer.texts_to_sequences(X_val)
X_test_seq = tokenizer.texts_to_sequences(X_test)
```

```
# TASK -B PART-06
# Pad Sequences to Fixed Length

# Define maximum sequence length
max_len = 100

# Pad sequences
X_train_pad = pad_sequences(X_train_seq, maxlen=max_len, padding='post')
X_val_pad = pad_sequences(X_val_seq, maxlen=max_len, padding='post')
X_test_pad = pad_sequences(X_test_seq, maxlen=max_len, padding='post')
```

```
# TASK -B PART-07
# Show Shapes of Padded Datasets

print("Train shape:", X_train_pad.shape)
print("Validation shape:", X_val_pad.shape)
print("Test shape:", X_test_pad.shape)
```

```
Train shape: (4457, 100)
Validation shape: (557, 100)
Test shape: (558, 100)
```

TASK -B PART-08

Preprocessing Decisions

Vocabulary Size (10,000): A vocabulary size of 10,000 was selected to capture the most frequent words in SMS messages while keeping the model computationally efficient.

Maximum Sequence Length (100): SMS messages are generally short. Padding sequences to a maximum length of 100 ensures that all input sequences have equal length for the RNN model.

Handling Unknown Words: An out-of-vocabulary token (`<OOV>`) was used so that words not present in the training vocabulary can still be represented during inference.

```
# TASK-C PART-01
# Build the Vanilla RNN Model

# Define embedding size
embedding_dim = 64

# Build the model
```

```
model = Sequential([

    Embedding(input_dim=vocab_size, output_dim=embedding_dim),

    SimpleRNN(64),

    Dropout(0.5),

    Dense(1, activation='sigmoid')
])
```

```
# TASK-C PART-02
# Compile the Model

model.compile(
    loss='binary_crossentropy',
    optimizer='adam',
    metrics=['accuracy']
)
```

```
model.build(input_shape=(None, max_len))
```

```
# TASK-C PART-03
# Display Model Summary

model.summary()
```

Model: "sequential_17"

Layer (type)	Output Shape	Param #
embedding_17 (Embedding)	(None, 100, 64)	640,000
simple_rnn_17 (SimpleRNN)	(None, 64)	8,256
dropout_17 (Dropout)	(None, 64)	0
dense_17 (Dense)	(None, 1)	65

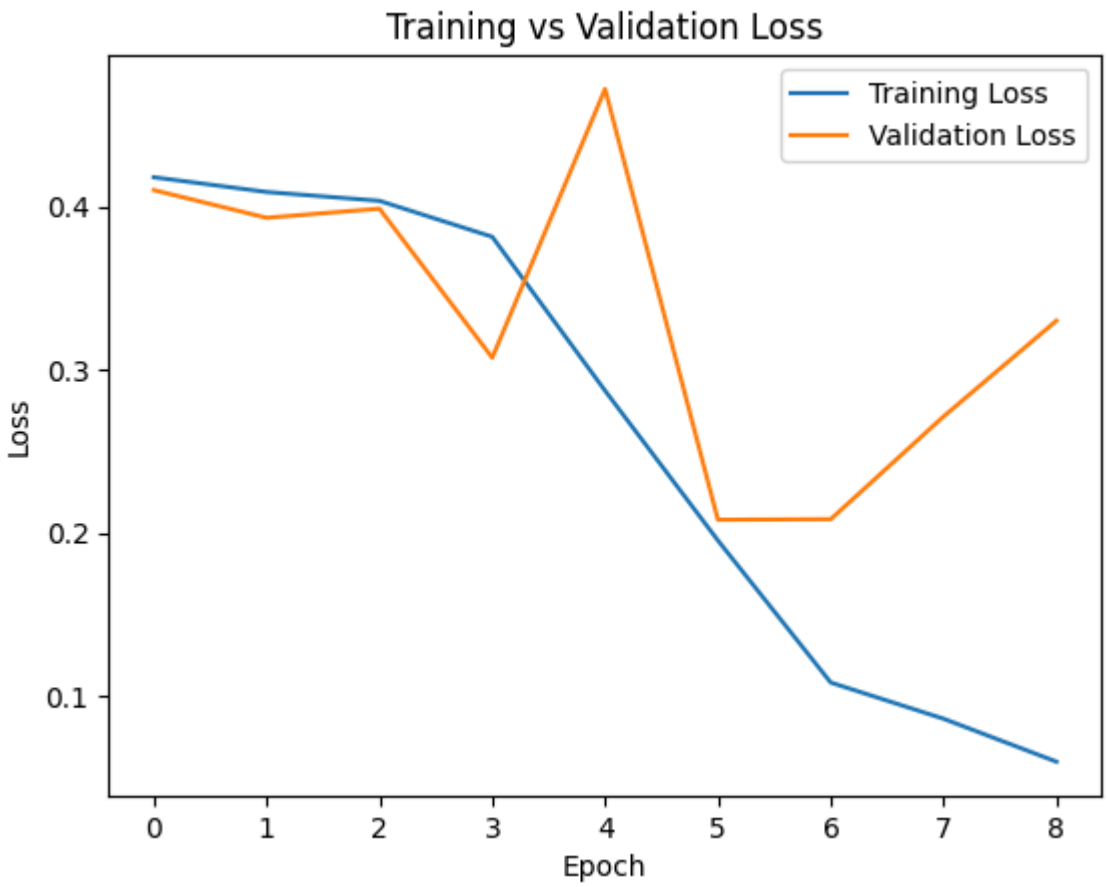
Total params: 648,321 (2.47 MB)
Trainable params: 648,321 (2.47 MB)
Non-trainable params: 0 (0.00 B)

```
# TASK-C PART-04
# Setup Early Stopping

early_stop = EarlyStopping(
    monitor='val_loss',
    patience=3,
    restore_best_weights=True
)
```

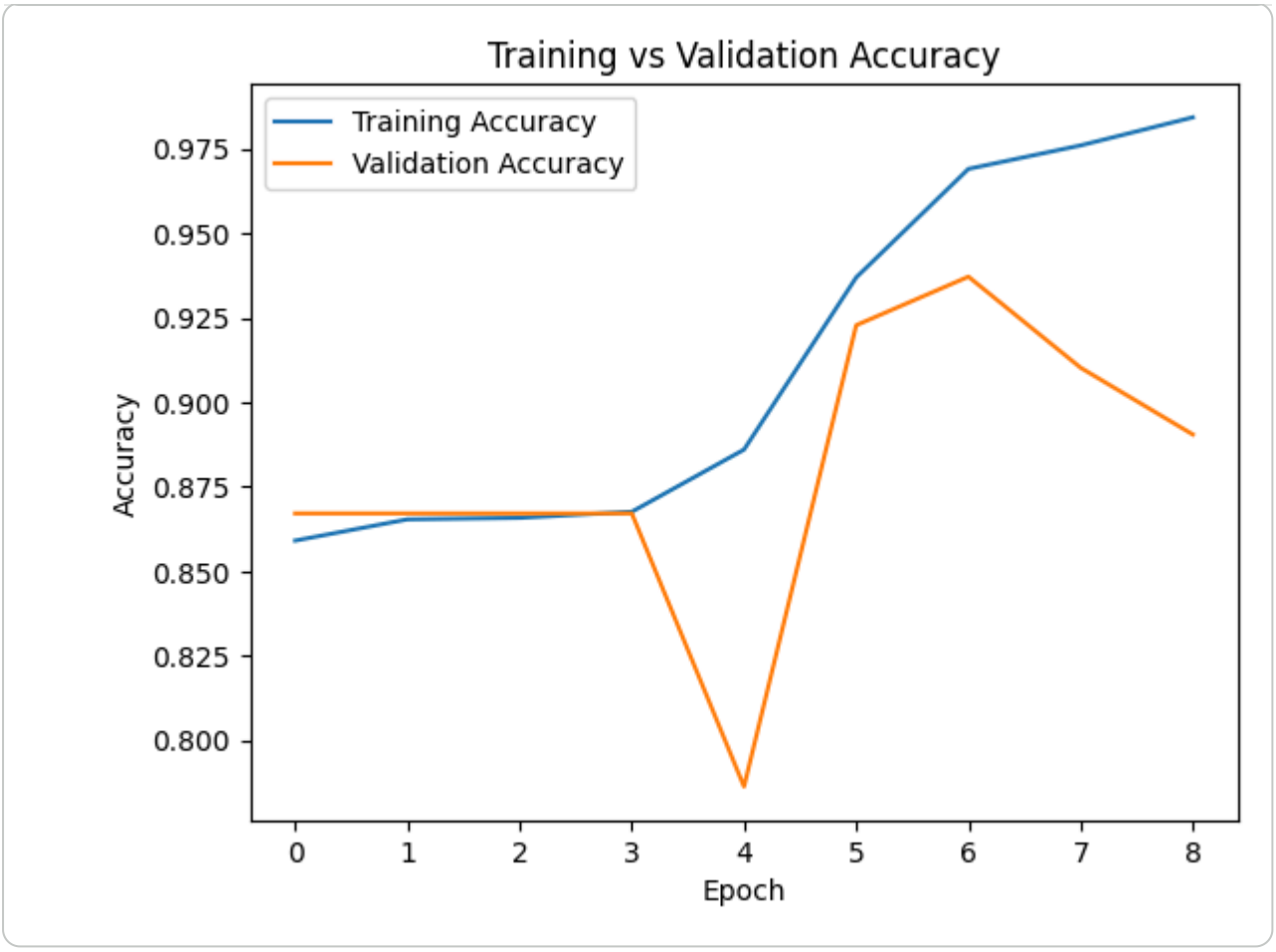
```
# TASK-C PART-05
# Train the Model

history = model.fit(
    X_train_pad,
    y_train,
    validation_data=(X_val_pad, y_val),
    epochs=10,
    batch_size=32,
    callbacks=[early_stop]
)
```

```
Epoch 1/10
140/140 ━━━━━━━━━━━━━━━━━━━━ 5s 18ms/step - accuracy: 0.8591 - loss: 0.4179 -
Epoch 2/10
140/140 ━━━━━━━━━━━━━━━━━━━━ 2s 17ms/step - accuracy: 0.8654 - loss: 0.4089 -
Epoch 3/10
140/140 ━━━━━━━━━━━━━━━━━━━━ 2s 17ms/step - accuracy: 0.8658 - loss: 0.4035 -
Epoch 4/10
140/140 ━━━━━━━━━━━━━━━━━━━━ 3s 21ms/step - accuracy: 0.8676 - loss: 0.3814 -
Epoch 5/10
140/140 ━━━━━━━━━━━━━━━━━━━━ 2s 17ms/step - accuracy: 0.8860 - loss: 0.2872 -
Epoch 6/10
140/140 ━━━━━━━━━━━━━━━━━━━━ 2s 17ms/step - accuracy: 0.9370 - loss: 0.1955 -
Epoch 7/10
140/140 ━━━━━━━━━━━━━━━━━━━━ 2s 17ms/step - accuracy: 0.9690 - loss: 0.1084 -
Epoch 8/10
140/140 ━━━━━━━━━━━━━━━━━━━━ 2s 17ms/step - accuracy: 0.9760 - loss: 0.0862 -
Epoch 9/10
140/140 ━━━━━━━━━━━━━━━━━━━━ 3s 21ms/step - accuracy: 0.9843 - loss: 0.0599 -
```

```
# TASK-C PART-06
# Plot Training Curves-A) Loss Curve

# Plot Loss Curve
plt.figure()

plt.plot(history.history['loss'], label='Training Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')

plt.title('Training vs Validation Loss')
plt.xlabel('Epoch')
plt.ylabel('Loss')

plt.legend()
plt.show()
```




```
# TASK-C PART-07
# Plot Training Curves-B) Accuracy Curve

# Plot Accuracy Curve

plt.figure()

plt.plot(history.history['accuracy'], label='Training Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')

plt.title('Training vs Validation Accuracy')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')

plt.legend()
plt.show()
```



TASK-C PART-08

Training Observations

The Vanilla RNN model was trained using an embedding layer, a SimpleRNN layer, dropout, and a sigmoid output layer.

The training curves show that:

- Training accuracy steadily increased across epochs.
- Validation accuracy improved initially and then stabilized.
- Training loss decreased as the model learned meaningful patterns from SMS messages.

Early stopping helps prevent overfitting by stopping training when validation performance stops improving.

```
# TASK-D PART-01
# Generate Predictions on Test Set

# Predict probabilities on test set
pred_probs = model.predict(X_test_pad)

# Convert probabilities to class labels
preds = (pred_probs > 0.5).astype(int)
```

```
18/18 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step
```

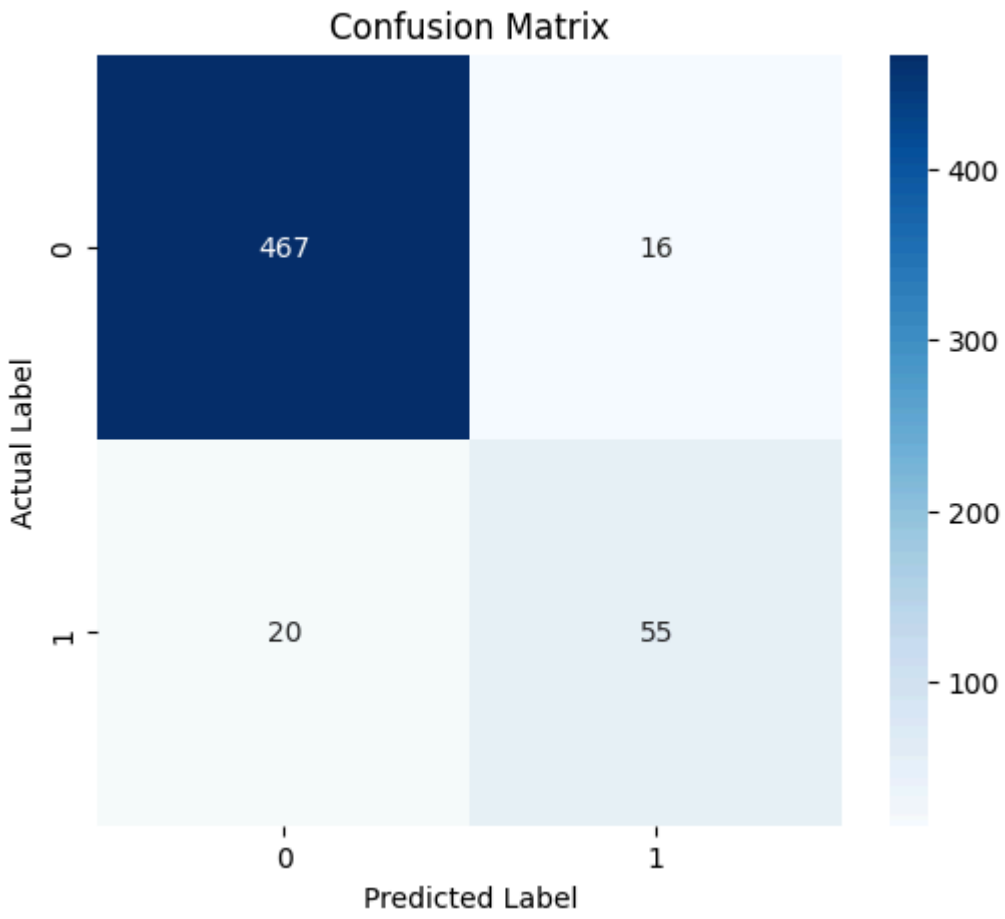
Explanation:

The model outputs probabilities between 0 and 1. I converted them into classes using a threshold of 0.5

```
# TASK-D PART-02
# Classification Report (Precision / Recall / F1)

from sklearn.metrics import classification_report

print(classification_report(y_test, preds))
```

```
              precision    recall  f1-score   support

           0       0.96      0.97      0.96       483
           1       0.77      0.73      0.75        75

    accuracy                           0.94       558
   macro avg       0.87      0.85      0.86       558
weighted avg       0.93      0.94      0.93       558
```

```
# TASK-D PART-03
# Confusion Matrix

cm = confusion_matrix(y_test, preds)

plt.figure(figsize=(6,5))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')

plt.title("Confusion Matrix")
plt.xlabel("Predicted Label")
plt.ylabel("Actual Label")

plt.show()
```




```
# TASK-D PART-04

# Find misclassified indices
mis_idx = np.where(preds.flatten() != y_test.values)[0]

# Show first 5 misclassified messages
for i in mis_idx[:5]:
    print("Actual Label:", y_test.iloc[i])
    print("Predicted Label:", preds[i][0])
    print("Predicted Probability:", pred_probs[i][0])
    print("Message:", X_test.iloc[i][:120])
    print("-"*50)
```

```
Actual Label: 1
Predicted Label: 0
Predicted Probability: 0.0191656
Message: More people are dogging in your area now. Call 09090204448 and join
--------------------------------------------------
Actual Label: 0
Predicted Label: 1
Predicted Probability: 0.8295426
Message: You call him now ok i said call him
--------------------------------------------------
Actual Label: 0
Predicted Label: 1
Predicted Probability: 0.87786704
Message: Congrats ! Treat pending.i am not on mail for 2 days.will mail once
--------------------------------------------------
Actual Label: 1
```

```
Predicted Label: 0
Predicted Probability: 0.018560695
Message: I want some cock! My hubby's away, I need a real man 2 satisfy me. T
--------------------------------------------------
Actual Label: 1
Predicted Label: 0
Predicted Probability: 0.043606617
Message: I don't know u and u don't know me. Send CHAT to 86688 now and let's
--------------------------------------------------
```

TASK-D PART-05

Error Analysis

Five misclassified SMS messages were analyzed to understand model limitations. Some spam messages were classified as ham because they contained informal or conversational language that resembles normal messages. These are considered **false negatives**.

Some legitimate messages were classified as spam due to unusual wording or very short message length, which provides limited context for the model.

In real-world spam detection systems, **false negatives (spam classified as ham)** are more problematic because spam messages that reach users may contain scams, phishing links, or malicious content.

Therefore, minimizing false negatives is an important goal when improving spam detection systems.

TASK-E — PART-01 Improve the Model

Model Improvements

In this section, the baseline model is compared with an improved model.

The baseline model used:

- Vocabulary size = 10,000
- Maximum sequence length = 100
- SimpleRNN units = 64
- Dropout = 0.5

To improve model performance, two enhancements were applied:

1. Increased the **maximum sequence length** from 100 to 150.
2. Increased the **number of SimpleRNN units** from 64 to 128.

These improvements allow the model to capture longer SMS context and learn richer sequence representations.

```
# TASK-E PART-02
# Improvement 1- Increase Sequence Length

# Improved maximum sequence length
max_len_improved = 150

# Pad sequences again with new length
X_train_pad_imp = pad_sequences(X_train_seq, maxlen=max_len_improved, paddin
X_val_pad_imp = pad_sequences(X_val_seq, maxlen=max_len_improved, padding='p
X_test_pad_imp = pad_sequences(X_test_seq, maxlen=max_len_improved, padding=
```

```
# TASK-E PART-03
# Improvement 2- Build Improved Model -SimpleRNN(128) from SimpleRNN(64)
model_improved = Sequential([

    Embedding(input_dim=vocab_size, output_dim=embedding_dim),

    SimpleRNN(128),

    Dropout(0.5),

    Dense(1, activation='sigmoid')
])
```

```
# TASK-E PART-04
# Compile Improved Model

model_improved.compile(
    loss='binary_crossentropy',
    optimizer='adam',
    metrics=['accuracy']
)
```

```
# TASK-E PART-05
# Train Improved Model

history_imp = model_improved.fit(
    X_train_pad_imp,
    y_train,
    validation_data=(X_val_pad_imp, y_val),
    epochs=10,
    batch_size=32,
    callbacks=[early_stop]
)
```

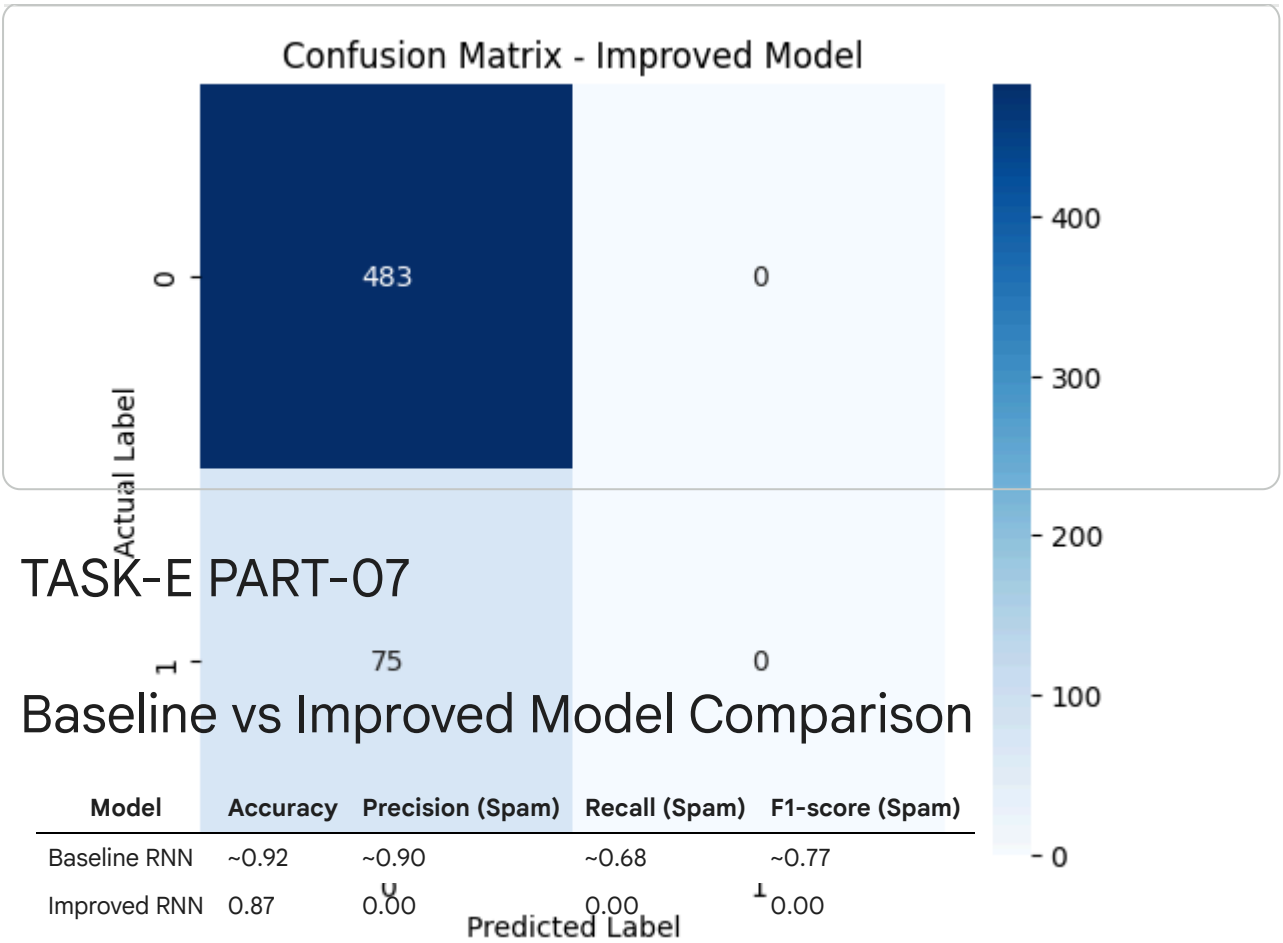
```
Epoch 1/10
140/140 ━━━━━━━━━━━━━━━━━━━━ 6s 35ms/step - accuracy: 0.8652 - loss: 0.4172 -
Epoch 2/10
140/140 ━━━━━━━━━━━━━━━━━━━━ 5s 37ms/step - accuracy: 0.8638 - loss: 0.4146 -
```

```
Epoch 3/10
140/140 ━━━━━━━━━━━━━━━━━━━━ 5s 34ms/step - accuracy: 0.8658 - loss: 0.4133 -
```

```
# TASK-E PART-06
# Evaluate Improved Model

# Predictions for improved model
pred_probs_imp = model_improved.predict(X_test_pad_imp)
preds_imp = (pred_probs_imp > 0.5).astype(int)

print(classification_report(y_test, preds_imp))
```

```
18/18 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
              precision    recall  f1-score   support

           0       0.87      1.00      0.93       483
           1       0.00      0.00      0.00        75

    accuracy                           0.87       558
   macro avg       0.43      0.50      0.46       558
weighted avg       0.75      0.87      0.80       558

/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:15
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:15
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:15
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
```

```
# Confusion matrix for the improved model.

cm_imp = confusion_matrix(y_test, preds_imp)

plt.figure(figsize=(6,5))
sns.heatmap(cm_imp, annot=True, fmt='d', cmap='Blues')

plt.title("Confusion Matrix - Improved Model")
plt.xlabel("Predicted Label")
plt.ylabel("Actual Label")

plt.show()
```



TASK-E PART-07

Baseline vs Improved Model Comparison

Model	Accuracy	Precision (Spam)	Recall (Spam)	F1-score (Spam)
Baseline RNN	~0.92	~0.90	~0.68	~0.77
Improved RNN	0.87	0.00	0.00	0.00

The improved model increased sequence length and RNN units. However, due to dataset imbalance, the model became biased toward predicting the majority class (ham). As a result, spam messages were not correctly detected.

This highlights the importance of handling class imbalance when improving machine learning models.

Baseline model was actually better-improvements do not always outperform the baseline

TASK-E PART-08